



Cairo Campus of



Cape
Breton
University



**An Efficient Car Rental System
to Meet Customer Needs
(Applied, Carenta)**



By

Name	ID
Mohamed Hany	202206511
Aya Gamal	202206147
Gerges Samir	202206437
Radwa Aboud	202206876
Tamer Ismail	202206879
Mariam Mostafa	202206938

Supervised by

Dr. Mohamed Ramadan

This project was submitted to the
Canadian International College in
partial fulfillment of the
requirements for the bachelor's
degree in the Department of
Business Information Systems.

2026



ABSTRACT

This project presents the development of a mobile application for car rental services that allows customers to rent cars in a fast, secure, and convenient way. The application is designed to connect car owners and rental providers with customers through a simple and user-friendly platform. The system offers a variety of vehicles, enabling users to select cars based on their preferences, budget, and availability. It also includes essential features such as user registration and authentication, real-time vehicle availability, advanced search and filtering, booking management, secure online payment, and customer reviews and ratings. In addition, the application supports role-based access for both customers and car owners, allowing efficient management of vehicles, bookings, and rental operations. By utilizing modern mobile technologies, the platform simplifies the traditional car rental process, improves service efficiency, and enhances user experience. The proposed system contributes to the digital transformation of transportation services by providing a reliable, scalable, and efficient solution for modern car rental management.



ACKNOWLEDGEMENTS

First and foremost, we would like to express our deepest gratitude and sincere appreciation to Allah Almighty for granting us the strength, patience, and perseverance necessary to successfully complete this graduation project. His continuous blessings and guidance have been the cornerstone of our ability to overcome challenges throughout this journey. We would like to extend our highest respect and profound gratitude to Dr. Mohamed Ramadan, our project supervisor, for his exceptional supervision, invaluable guidance, and unwavering support throughout all stages of this work. His insightful feedback, constructive criticism, and continuous encouragement have been instrumental in shaping both the direction and the quality of this project. We are also deeply grateful to Dr. Dalia Sayed and Dr. Manar Mahmoud for their continuous academic support, encouragement, and valuable input, which have significantly contributed to enhancing our understanding and improving the overall standard of this work. Finally, we would like to express our sincere appreciation to our families and friends for their unconditional love, patience, and continuous encouragement. Their constant moral support and belief in our abilities have been a vital source of motivation that enabled us to persist through difficulties and successfully complete this work.



Table of content

Chapter One General Framework of the Study.....	9
1.1 Introduction:	10
1.2 Problem definition.....	11
1.3 Existing System	12
1.4 Purpose System	13
1.5 Motivation of the work	14
1.6 Study Variables	15
1.7 Objectives:	16
1.8 Study importance	18
1.9 Methodology	19
1.10 Application tools.....	21
1.11 Executive Summary.....	23
1.12 Purpose Application.....	23
1.13 Thesis Structure.....	25
Chapter 2 Car Rental System.....	26
2.1 Car Rental System Performance.....	27
2.2 Evaluating User Satisfaction.....	29
Chapter 3 System Analysis and Design.....	32
3.1 System Analysis.....	32
3.1.1 Functional requirements:.....	33
3.1.1.1 Renter Functional requirements:.....	33
3.1.1.2 Owner Functional requirements:.....	33



3.1.2 Non-functional requirements.....	34
3.1.3 Use Case Diagram	35
3.1.4 Class Diagram.....	36
3.1.5 Sequence diagram.....	37
3.1.6 Activity diagram.....	40
3.1.7 Context diagram.....	42
3.1.8 Data Flow Diagram.....	43
3.1.9 Use Case Description.....	44
System Design	68
3.2.1 Entity Relationship Diagram(ERD).....	67
3.2.2 Mapping Schema:.....	68
Chapter Four System Implementation.....	69
4.1 UI Screens:.....	70
4.2 System Code	75
Chapter Five Conclusion and Future Work.....	78
5.1 Conclusion:.....	79
5.2 Future work.....	80



List of Figures

Figure 1: Use Case Diagram.....	35
Figure 2: Class Diagram	36
Figure 3 Sequence Diagram – request & approval ,payment, booking.....	37
Figure 4 Sequence Diagram – search & rental	38
Figure 5 Sequence Diagram Authentication	38
Figure 6 Sequence Diagram – owner car management	39
Figure 7: Activity Diagram - Renter activity	40
Figure 8: Activity Diagram – Owner activity	41
Figure 9: Context Diagram	42
Figure 10: Data Flow Diagram	43
Figure 11: Entity Relationship Diagram.....	67
Figure 12: Mapping Schema.....	69
Figure 13: Splash Screen	70
Figure 14: Choose Role Screen.....	70
Figure 15: Login Screen.....	71
Figure 16: Rental Details Screen.....	71
Figure 17: Renter Home Page Screen.....	72
Figure 18: My Cars Screen.....	72
Figure 19: Add New Car Screen.....	73
Figure 20: Owner Request Screen.....	73
Figure 21: Owner Home Screen.....	74
Figure 22: Owner Profile Screen	74
Figure 23: Car description Page Code	75
Figure 24: My Rentals Page Code	75



Figure 25: Add Car Page Code	76
Figure 26: Authentication Page Code	76
Figure 27: Pubspec.yaml Page Code	77
Figure 28: Main Page Code	77



CHAPTER I

General Framework of the Study



Chapter One:

1.1 introduction

In recent years, digital applications and online services have become an essential part of everyday life, particularly in the transportation sector. The increasing demand for flexible mobility solutions has led many individuals to rely on car rental services, whether for short-term use such as daily commuting or quick trips, or for longer durations, especially in the case of tourists who require convenient transportation throughout their stay to explore destinations freely without depending on public transport.

Traditionally, car rental processes were manual and required physical visits, paper-based documentation, and direct communication between customers and service providers. These methods were often time-consuming, prone to errors, and inefficient in managing bookings and availability. With the advancement of technology, the industry has gradually shifted toward digital platforms that provide more efficient, reliable, and user-friendly solutions.

Modern vehicle rental systems now enable users to browse available cars, view detailed information, and complete booking processes in a more convenient and organized manner. These systems have significantly improved service accessibility and operational efficiency by reducing manual effort and minimizing delays.

Although digital transformation has greatly enhanced the car rental industry, certain challenges still exist, such as the need for real-time updates, system reliability, and ensuring a smooth user experience across different user categories. Therefore, developing an efficient and integrated car rental system remains essential to further improve accessibility, flexibility, and overall service quality.



1.2 problem Definition

Customers currently face several challenges when attempting to rent a car through traditional or partially digital rental systems. One of the main issues is fragmented search, where users are required to browse multiple applications, websites, or rental providers in order to find a suitable vehicle that meets their needs, which leads to inefficiency and wasted time.

Another significant limitation is the lack of flexibility in rental durations. Many existing systems offer restricted or rigid rental options that do not adequately support both short-term requirements, such as daily commuting or short trips, and long-term needs, such as extended tourist stays or long-duration usage. This reduces the adaptability of rental services to different user categories.

In addition, pricing transparency remains a major concern. Users often encounter unclear pricing structures, hidden charges, or difficulty in comparing different rental offers, which negatively affects trust and decision-making. Furthermore, the overall user experience in many systems is often inefficient, requiring time-consuming procedures and delayed access to services.

There is also a noticeable limitation in supporting long-term rental options, which particularly impacts tourists and users who require continuous vehicle availability for extended periods.

Core Problem Statement

There is no unified and efficient car rental system that provides transparent pricing, flexible rental duration options, and a seamless user experience for both short-term and long-term customers, while ensuring easy access to reliable and real-time vehicle availability.



1.3 Existing System

The traditional car rental system is primarily based on manual processes, where customers are required to visit rental offices physically to inquire about available vehicles, pricing, and rental conditions. This approach often relies on paper-based documentation and direct communication between customers and service providers.

In such systems, vehicle availability is usually updated manually, which increases the possibility of errors, delays, and outdated information. Customers may also face difficulties in comparing different car options, as information is not always organized or easily accessible in one place. Additionally, the booking process can be time-consuming, as it involves multiple steps including verification, paperwork, and confirmation.

Some modern rental companies have adopted partial digital solutions, such as basic websites or phone-based reservation systems. However, these systems are often limited in functionality and do not provide real-time updates or seamless user experiences. They may also lack proper integration between users, car availability, and booking management.

Overall, the existing systems suffer from limitations such as inefficiency, lack of real-time data, and dependency on manual operations, which reduce user satisfaction and highlight the need for more advanced and integrated digital solutions.



1.4 Proposed system

A unified and efficient car rental platform designed to address the limitations of traditional and existing rental solutions. It provides a centralized digital environment that connects car owners with customers, enabling seamless interaction and improved accessibility to rental services.

The system allows users to easily browse available vehicles, view detailed information, and check real-time availability, ensuring accurate and up-to-date data at all times. It also supports flexible rental options that accommodate both short-term and long-term requirements, making it suitable for daily users as well as tourists.

In addition, the system enhances transparency by providing clear pricing information and allowing users to compare different vehicles before making a decision. A role-based structure is implemented to distinguish between users and car owners, ensuring proper access control and functionality for each type of user.

Furthermore, the system improves overall user experience through a simple and intuitive interface, real-time data synchronization, and efficient data management. Cloud-based storage is used for handling vehicle images, while modern backend services ensure scalability and reliability.

Overall, the proposed system aims to provide a more organized, transparent, and user-friendly car rental experience compared to existing solutions.



1.5 Motivation of the Work

The motivation behind this project arises from the increasing demand for efficient, flexible, and accessible transportation solutions in modern society. With the rapid growth of urban populations and tourism activities, there is a continuous need for convenient car rental services that can accommodate different user requirements, whether for short-term daily use or long-term stays.

Traditional car rental systems often suffer from several limitations, such as time-consuming procedures, lack of real-time information, limited transparency in pricing, and inefficient booking processes. These challenges create a gap between customer expectations and the actual services provided, leading to reduced satisfaction and inconvenience for users.

In addition, the shift toward digital transformation in various industries has highlighted the importance of developing smart and integrated systems that improve service efficiency and user experience. This project is motivated by the need to bridge this gap by providing a unified car rental platform that simplifies the rental process, enhances transparency, and supports real-time access to available vehicles.

Furthermore, the project is inspired by the goal of improving accessibility for both local users and tourists by offering flexible rental options and a more organized system compared to traditional methods. Overall, this work aims to contribute to the enhancement of the car rental industry through the application of modern software technologies.



1.6 Study Variables:

Independent Variables (Inputs / Causes)

These are the factors the system manipulates or that influence the rental process:

- **Rental duration type** (short-term vs. long-term)
- **Vehicle availability** (real-time inventory status)
- **Pricing structure** (dynamic pricing, duration-based rates)
- **User search criteria** (vehicle type, date range, location)
- **Platform usability features** (interface design, booking steps)

Dependent Variables (Outputs / Effects)

These are the outcomes affected by the independent variables:

- **Customer satisfaction** (ease of use, transparency, flexibility)
- **Booking efficiency** (time spent searching and comparing)
- **Rental flexibility** (ability to find suitable long-term or short-term options)
- **User experience quality** (convenience, reduced frustration)
- **System adoption rate** (customer trust and repeated use)



1.7 Objectives:

General Objective

The main objective of this project is to design and develop an efficient and integrated car rental system that effectively meets the diverse needs of customers by supporting both short-term and long-term rental requirements. The system aims to provide a modern digital solution that improves accessibility, convenience, and overall service quality compared to traditional rental methods.

Specific Objectives

1. Reducing search and booking time

This means that instead of searching across multiple platforms or rental providers, the system integrates all available cars in one place. This helps users save time and effort when searching for and booking a vehicle.

2. Improving rental flexibility

This objective focuses on allowing users to choose rental durations that suit their needs, such as daily, weekly, or monthly rentals, instead of being restricted to fixed or limited options as in traditional systems.

3. Enhancing pricing transparency

This means that all rental prices are clearly displayed from the beginning without hidden charges. It also allows users to easily compare different cars and prices before making a decision.



4. User-friendly interface

This refers to designing a simple and easy-to-use interface that allows all types of users, even non-technical ones, to navigate the system smoothly and complete actions such as searching and booking without difficulty.

5. Supporting tourists and long-term users

This objective ensures that the system is not limited to short-term use only, but also supports tourists and users who require vehicles for extended periods without unnecessary restrictions.

6. Increasing customer satisfaction

This is the overall goal of all the previous objectives, where improving speed, flexibility, transparency, and usability leads to a better user experience and higher customer satisfaction compared to traditional rental methods.



1.8 Study Importance:

For Customers (End Users)

- ❖ Saves time and effort by eliminating repeated requests across multiple providers.
- ❖ Provides flexible rental durations tailored to individual needs (commuting vs. travel).
- ❖ Offers transparent pricing and easy vehicle comparison.

For Tourists

- ❖ Enable long-term rentals that match their stay duration.
- ❖ Reduces reliance on public transportation.
- ❖ Enhance travel, freedom and comfort.

For Rental Providers

- ❖ Increase vehicle utilization through better demand matching.
- ❖ Attracts more customers via an efficient, modern platform.
- ❖ Reduces administrative overhead from manual booking processes.

For the Rental Industry

- ❖ Sets a benchmark for digital transformation in car rental services.
- ❖ Demonstrates how integrated systems can solve fragmentation issues.
- ❖ Contributes to more organized, efficient, and customer-centric mobility services.



1.9 Methodology:

Given the nature of the project (a software/system development), the following methodology is recommended:

Overall Approach

Agile Development Methodology (Scrum or Kanban) – iterative delivery with continuous user feedback.

Phases:

PHASE	ACTIVITIES
1. REQUIREMENTS ANALYSIS	Interviews/surveys with customers and rental providers; identify functional and non-functional requirements.
2. SYSTEM DESIGN	UML diagrams (use case, class, sequence); database design; user interface wireframes.
3. DEVELOPMENT	Frontend (Flutter), Backend (e.g., Node.js, Django), Database (MySQL/PostgreSQL).
4. INTEGRATION	Connect to third-party services (payment gateway, SMS/email notifications, map APIs).
5. TESTING	Unit testing, integration testing, user acceptance testing (UAT).



6. DEPLOYMENT & EVALUATION	Pilot launch with real users; collect feedback via questionnaires; measure efficiency gains (time saved, satisfaction scores).
----------------------------	--

Data Collection Methods

Quantitative: System logs (search time, booking completion rate), user satisfaction surveys (Likert scale).

Qualitative: User interviews, focus groups with tourists and daily commuters.

Evaluation Metrics

Average time to find and book a vehicle (target: < 3 minutes)

User satisfaction score (target: > 4.5/5)

Rental duration flexibility index (number of available duration options)

System usability scale (SUS) score



1.10 Application Tools

1.10.1 FlutterFlow

FlutterFlow is a visual development platform used for building mobile applications based on the Flutter framework. It enables developers to design user interfaces using a drag-and-drop approach instead of writing code manually. It also allows the configuration of application logic through a visual interface, making it easier to define user interactions and workflows. Flutter Flow is particularly useful for rapid development and for users who prefer a visual approach to application design.

1.10.2 Google Firebase

Google Firebase is a cloud-based backend platform that provides a wide range of services for mobile and web applications. These services include authentication, real-time databases, cloud storage, and analytics. Firebase supports scalable application development and ensures efficient data management and synchronization between users in real time.

1.10.3 Microsoft Visio

Microsoft Visio is a diagramming tool used to create flowcharts, system diagrams, and various types of visual representations. It is commonly used in software engineering projects to design system workflows, data flow diagrams (DFDs), and architectural diagrams, helping in planning and documenting system structure.



1.10.4 ChatGPT

ChatGPT is an AI-based language model developed by OpenAI. It is capable of generating human-like text, answering questions, assisting in writing, and supporting content creation. In this project, it can be used as a supportive tool for documentation and improving the quality of written content.

1.10.5 Figma

Figma is a cloud-based UI/UX design and prototyping tool used for creating and collaborating on interface designs. It allows multiple users to work together in real time, making it effective for designing application layouts, user flows, and interactive prototypes

1.10.6 Supabase

Supabase is an open-source backend-as-a-service platform that provides scalable backend functionality for modern applications. It offers services such as database management, authentication, and file storage. In this project, Supabase is used primarily for storing and managing vehicle images efficiently, ensuring secure and reliable cloud storage. It simplifies backend integration and supports real-time data handling, making it a suitable solution for managing media assets within the application.



1.11 Executive Summary

This project presents the development of a mobile-based car rental system designed to simplify and enhance the process of renting vehicles. The application aims to solve common problems in traditional rental systems such as time consumption, lack of availability transparency, and limited flexibility in booking.

The system allows users to browse available cars, view detailed information and images, and book vehicles for specific durations with ease. It also supports real-time car availability, ensuring that users receive updated information instantly. In addition, the platform includes role-based access for customers and car owners, enabling efficient management of listings and bookings.

Modern technologies such as Flutter for mobile development, Firebase Firestore for real-time data synchronization, and cloud storage for image handling were used to ensure scalability, performance, and reliability.

Overall, this project provides a convenient, fast, and user-friendly solution for car rentals, improving both customer experience and resource utilization for rental providers.

1.12 previous Applications

1.12.1Turo

Turo is a peer-to-peer car sharing application that allows car owners to rent their vehicles to other users through a mobile platform. The application provides an easy way for customers to browse available cars, compare prices, select rental periods, and complete bookings online. It also enables owners to manage their vehicles, availability, and rental requests efficiently.

1.11.2 Getaround



is a car sharing application that connects vehicle owners with users who need temporary transportation. The platform allows customers to search for nearby cars, reserve them for specific durations, and unlock vehicles using the mobile application. The system focuses on providing flexible and convenient transportation services while simplifying the rental process.

1.12.3 Zipcar

Zipcar is a car sharing service that offers short-term vehicle rentals for users through a mobile application. Customers can locate available cars, make reservations, and access vehicles for hourly or daily use. The application aims to provide affordable and convenient transportation solutions while reducing the need for personal car ownership.

The following table shows a comparison between CARENTA application and the previous application:

Feature					
Usability	Medium	High	Low-Medium	High (simplified UI)	
Availability	Medium	High	Medium	High (real-time + simple)	
Scalability	High	High	High	Medium-High (for growth)	
System Complexity	High	High	Medium	Low (simplified design)	
Booking Process	Medium	Fast	Structured	Very Fast & Simple	
User Experience	Good	Very Good	Average	Very Good (clean & modern)	
Learning Curve	Medium	Low	Medium	Very Low (easy to use)	
Flexibility	High	Medium	Medium	High (custom features)	







2.1 Car Rental System Performance:

System performance is an important factor in the success of the Car Rental System because it determines how efficiently and reliably the system operates. A high-performance system improves user experience, increases operational efficiency, and ensures that rental processes are completed accurately and quickly.

The system is designed to handle different operations such as vehicle management, booking, payment processing, and customer information management in an efficient and organized way. The following factors explain the performance of the system in detail.

Fast Processing Speed

The system provides fast processing for different operations including searching for vehicles, making bookings, updating rental records, and processing payments. Quick response time helps users' complete tasks efficiently without unnecessary delays, which improves customer satisfaction.

Efficient Data Management

The system uses a well-structured database to store and manage information related to vehicles, renters, owners, bookings, and payments. Proper database organization helps retrieve and update data quickly and accurately while reducing data redundancy and errors.

Accuracy and Reliability

System performance depends on the accuracy of stored information and the reliability of operations. The system ensures that booking records, payment

Details, and vehicle availability are updated correctly to avoid conflicts or incorrect transactions.



Reduced Manual Work

The automated system minimizes manual paperwork and reduces the need for traditional rental procedures. This improves operational efficiency and decreases the possibility of human errors in booking and payment management.

Multi-User Support

The system can support multiple users at the same time, including renters, owners, and administrators. Different users can access the system simultaneously without significantly affecting system speed or performance.

Booking and Payment Efficiency

The booking and payment modules are designed to operate smoothly and efficiently. Users can reserve vehicles, confirm bookings, and complete payments in a simple and organized manner. Efficient transaction handling improves the overall rental process.

Scalability and Flexibility

The system is flexible enough to support future improvements and additional features. New vehicle types, services, or modules can be added without affecting the overall structure of the system. This scalability helps maintain good performance as the system grows.

Security Performance

The system also focuses on maintaining secure operations while protecting user and payment data. Secure data handling increases user trust and helps maintain reliable system performance.



Administrative Performance

Administrators can efficiently manage vehicles, bookings, renters, owners, and payments through organized management functions. The system simplifies administrative tasks and improves monitoring and decision-making processes.

To better understand the structure and functionality of the system, the following diagrams are presented in the next sections

Evaluating User Satisfaction

Evaluating user satisfaction is an essential process in the Cars Rental System because it helps determine whether the system successfully meets the needs and expectations of its users. User satisfaction reflects the quality, usability, and efficiency of the system and plays an important role in improving the overall rental service.

The system is designed to provide a smooth and convenient experience for renters, owners, and administrators. Therefore, user satisfaction is evaluated based on several important factors.

Ease of Use

One of the main factors affecting user satisfaction is the simplicity of the system interface. Users should be able to navigate the system easily without confusion. The booking process, payment process, and vehicle search functions should be clear and user-friendly. A simple and organized interface help users complete tasks quickly and efficiently.

System Performance and Speed

The speed of the system greatly influences user experience. Users expect fast responses when searching for vehicles, making bookings, or processing payments. A high-



performance system reduces waiting time and improves customer confidence and satisfaction.

Accuracy of Information

The system must provide accurate and updated information about available vehicles, rental details, booking status, and payments. Incorrect or outdated Information may lead to user dissatisfaction and operational problems. Therefore, maintaining accurate data is important for system reliability.

Reliability and Availability

Users expect the system to work properly without frequent errors or downtime. A reliable system ensures that users can access services at any time and complete rental operations successfully. System stability increases user trust and improves overall satisfaction.

Security and Privacy

Protecting user information is another important factor in evaluating satisfaction. The system should securely manage customer data and payment information. Users feel more comfortable using the system when they trust that their personal information is protected.

Booking and Payment Experience

The booking and payment processes should be smooth and efficient. Users should be able to reserve vehicles and complete payments with minimal effort. Simplifying these operations improves customer experience and increases satisfaction levels.



Customer Feedback and Reviews:

User satisfaction can also be evaluated through customer feedback, surveys, and reviews. Collecting user opinions helps identify system strengths and weaknesses. Feedback can be used to improve system features and enhance service quality in future updates.

Administrative Efficiency:

The system should also satisfy administrators by helping them manage vehicles, bookings, renters, owners, and payments efficiently. Easy Management functions reduce administrative workload and improve operational performance.

Conclusion:

Overall, evaluating user satisfaction helps measure the effectiveness and success of the Cars Rental System. By focusing on usability, performance, reliability, security, and service quality, the system can provide better rental experience and achieve higher levels of customer and administrator satisfaction.



System Analysis and Design



3.1 Functional Requirements

3.1.2 Renter Functional Requirements

- register an account
- log into application
- recover forgotten passwords
- view available cars
- search for cars
- view car details
- select a car for rental
- choose a rental plan
- submit a rental request
- make payments
- choose a payment method
- submit rental extension requests.

3.1.2 Owner Functional Requirements

- rental requests.
- accept rental requests.
- reject rental requests.
- add cars.
- update car information.
- remove cars.
- view bookings.
- view earnings.
- view rental history



3.2 Nonfunctional Requirements (System Qualities)

Security:

User data must be protected (encryption for passwords & payments)
Secure authentication and authorization
Safe payment processing

Performance:

Fast response when searching or viewing cars
System should handle multiple users at the same time

Usability:

Easy-to-use interface for renters and owners
Simple navigation between features

Scalability:

System should support increasing number of users and cars

Reliability:

System should be available with minimal downtime
Transactions (booking/payment) must not fail or lose data

Maintainability:

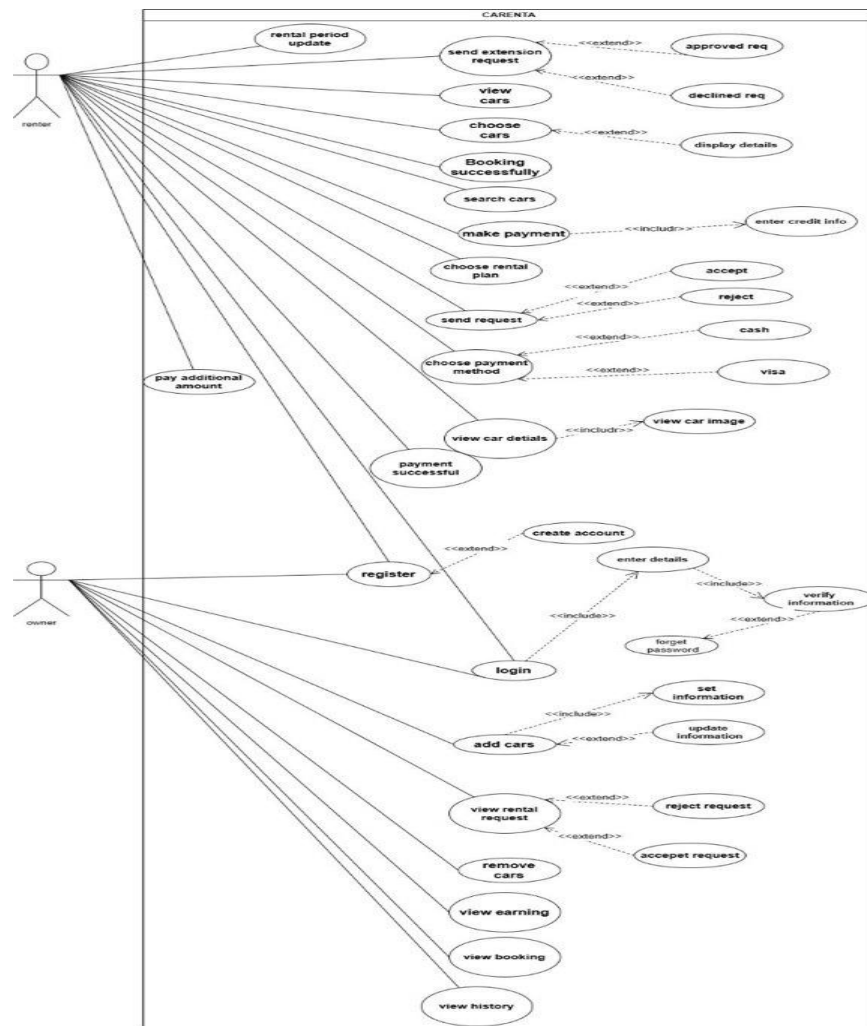
Easy to update or fix bugs
Clean and organized code structure

Compatibility:

Works on mobile and web platforms
Supports different browsers

3.3 Use Case Diagram:

A use case diagram is a visual representation that depicts the interactions between actors (users or external systems) and a system. It illustrates how users interact with the system to achieve specific goals or perform certain tasks.

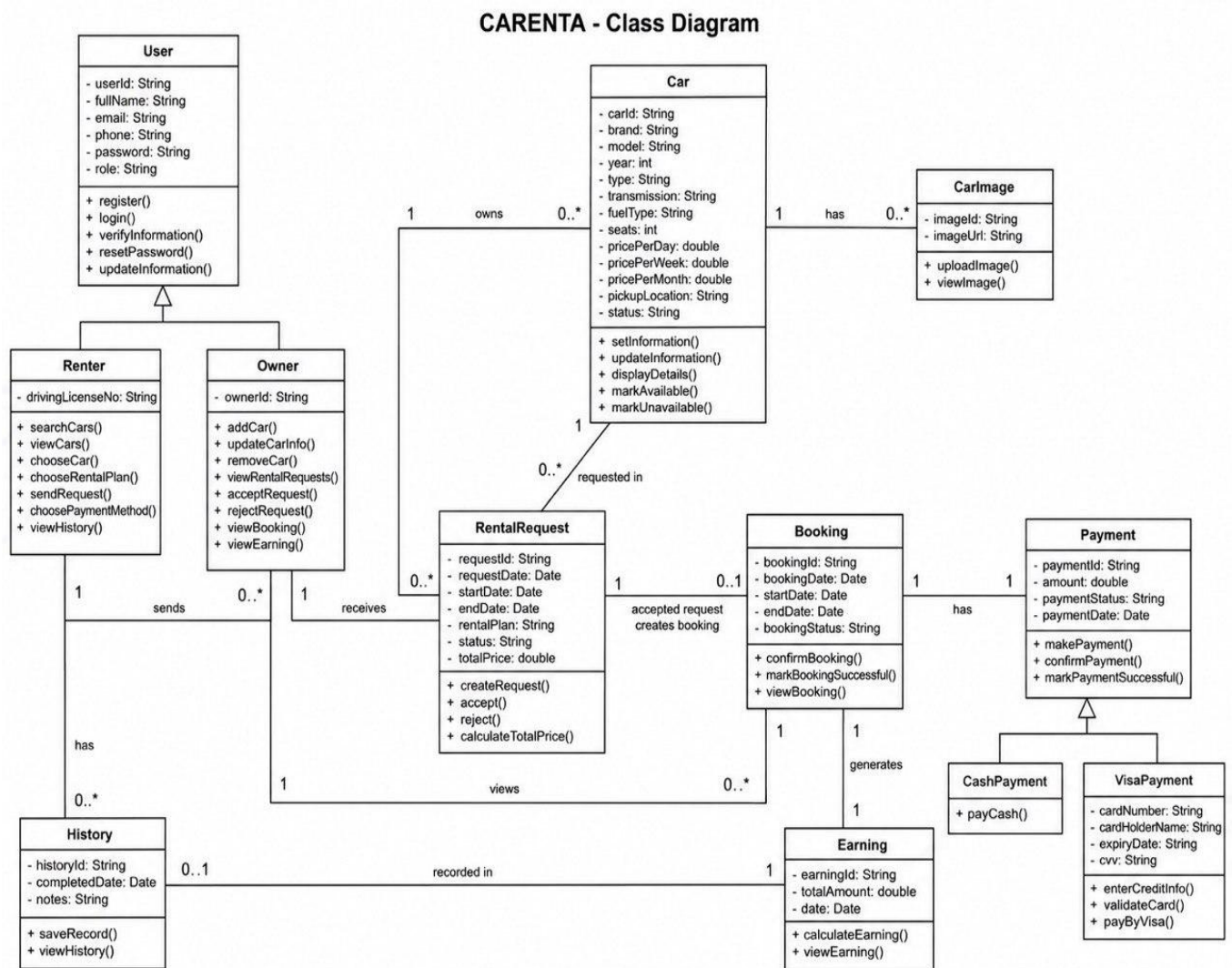


3.4 Class Diagram

This class diagram shows the car rental system. It presents the main classes: user, owner, renter, car, booking, payment, and history.

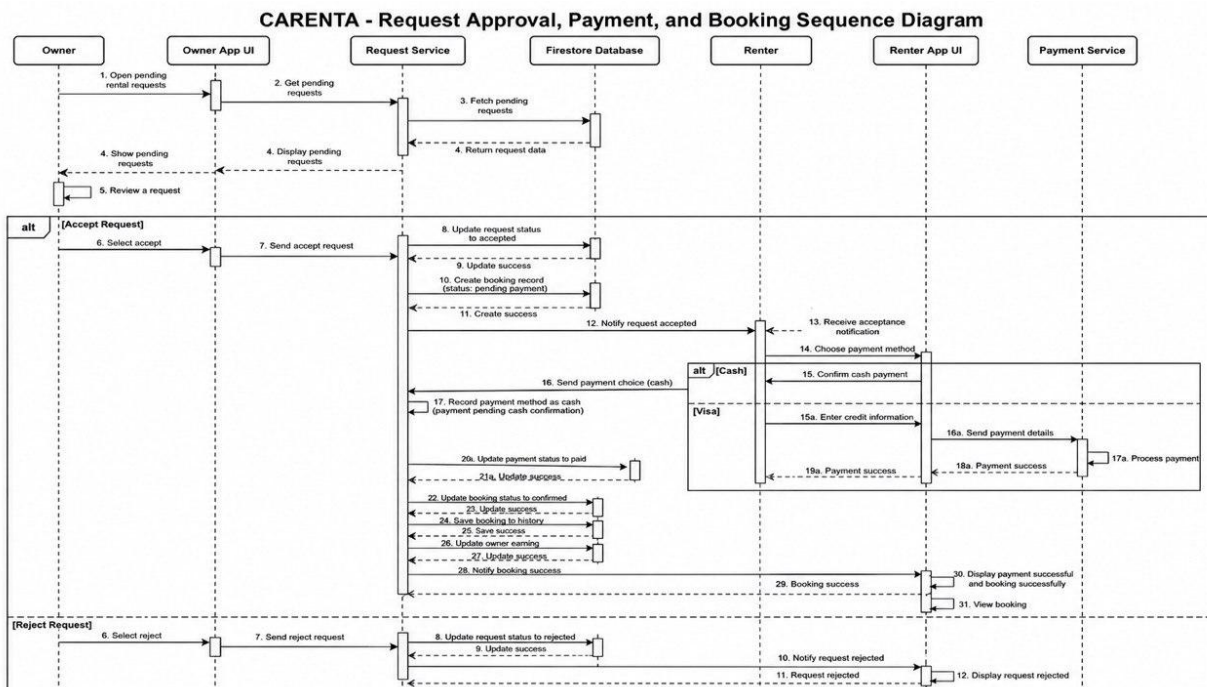
It also shows the attributes and methods of each class, along with their relationships.

The diagram is used to organize and visualize how the car rental process works.

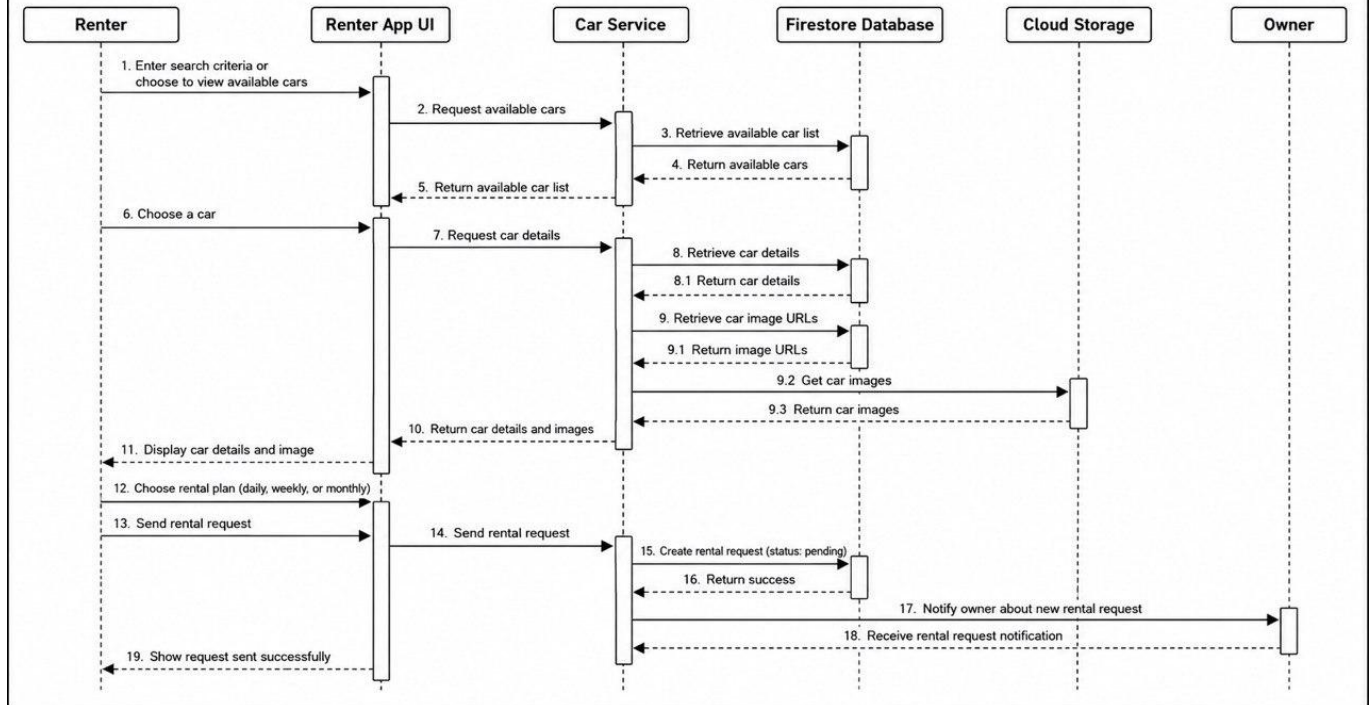


3.5 Sequence Diagram

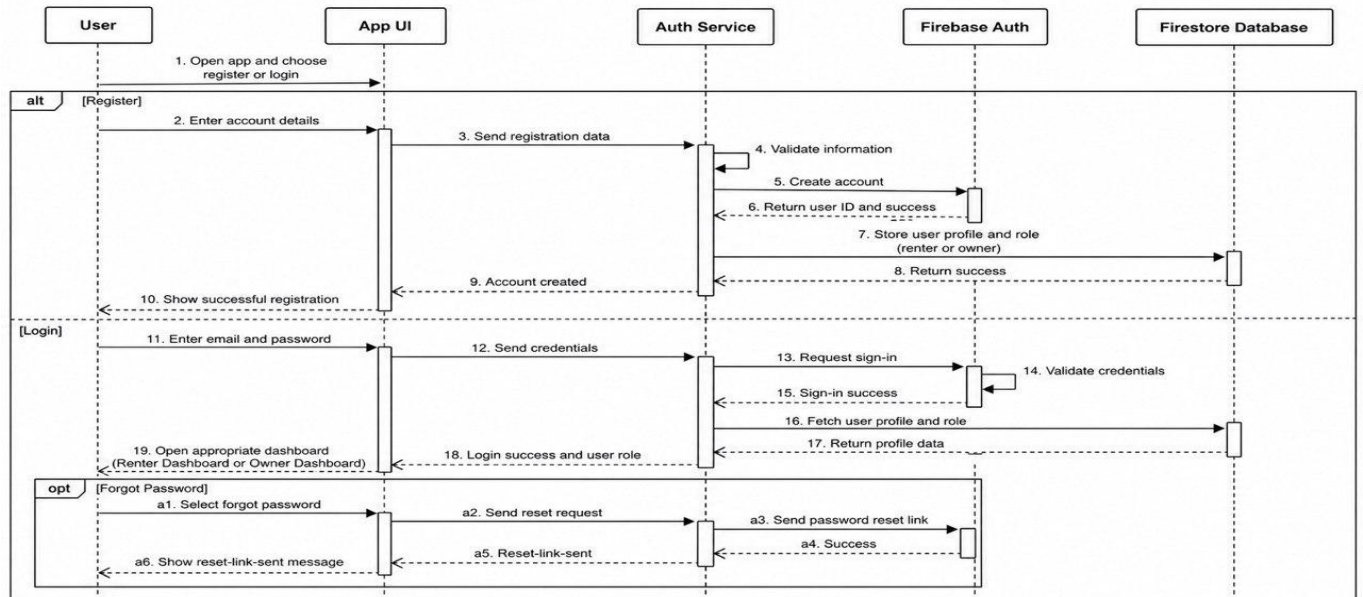
This sequence diagram illustrates the process flow for request approval, payment, and booking in CARENTA. The owner reviews and accepts a pending rental request, prompting the system to create a booking record with a "pending payment" status and notify the renter. The renter then selects a payment method (cash or Visa) and confirms the payment, after which the system updates the booking status to "confirm." If the owner rejects the request, the process terminates immediately without creating a booking or sending a payment notification.



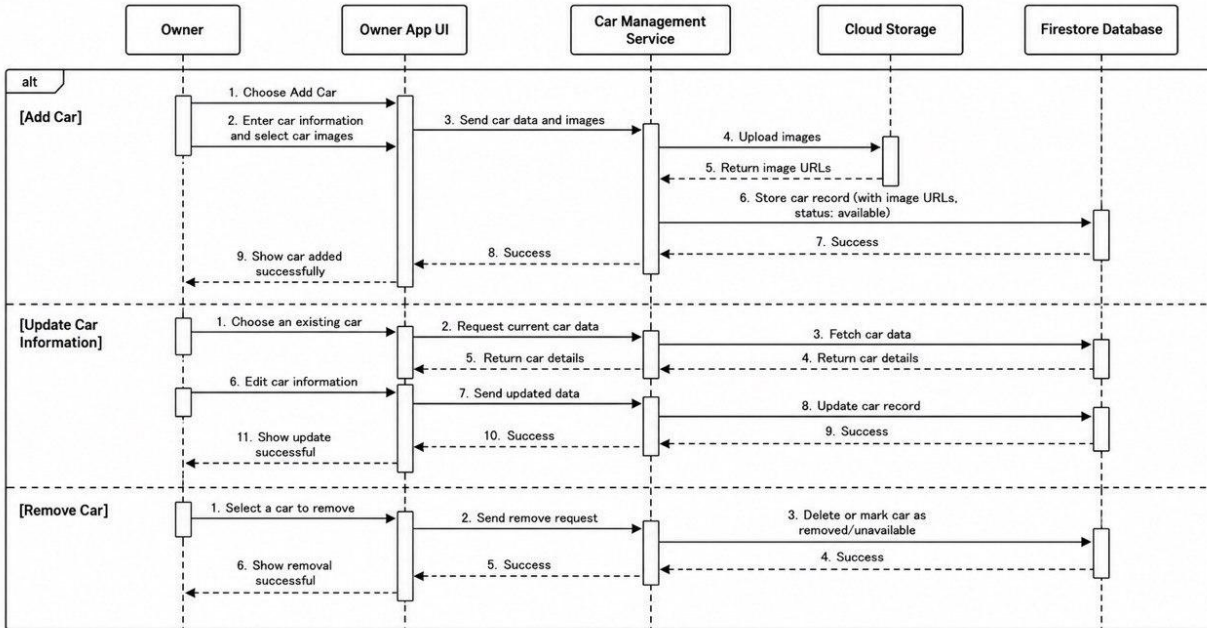
CARENTA - Renter Car Search and Rental Request Sequence Diagram



CARENTA - Authentication Sequence Diagram



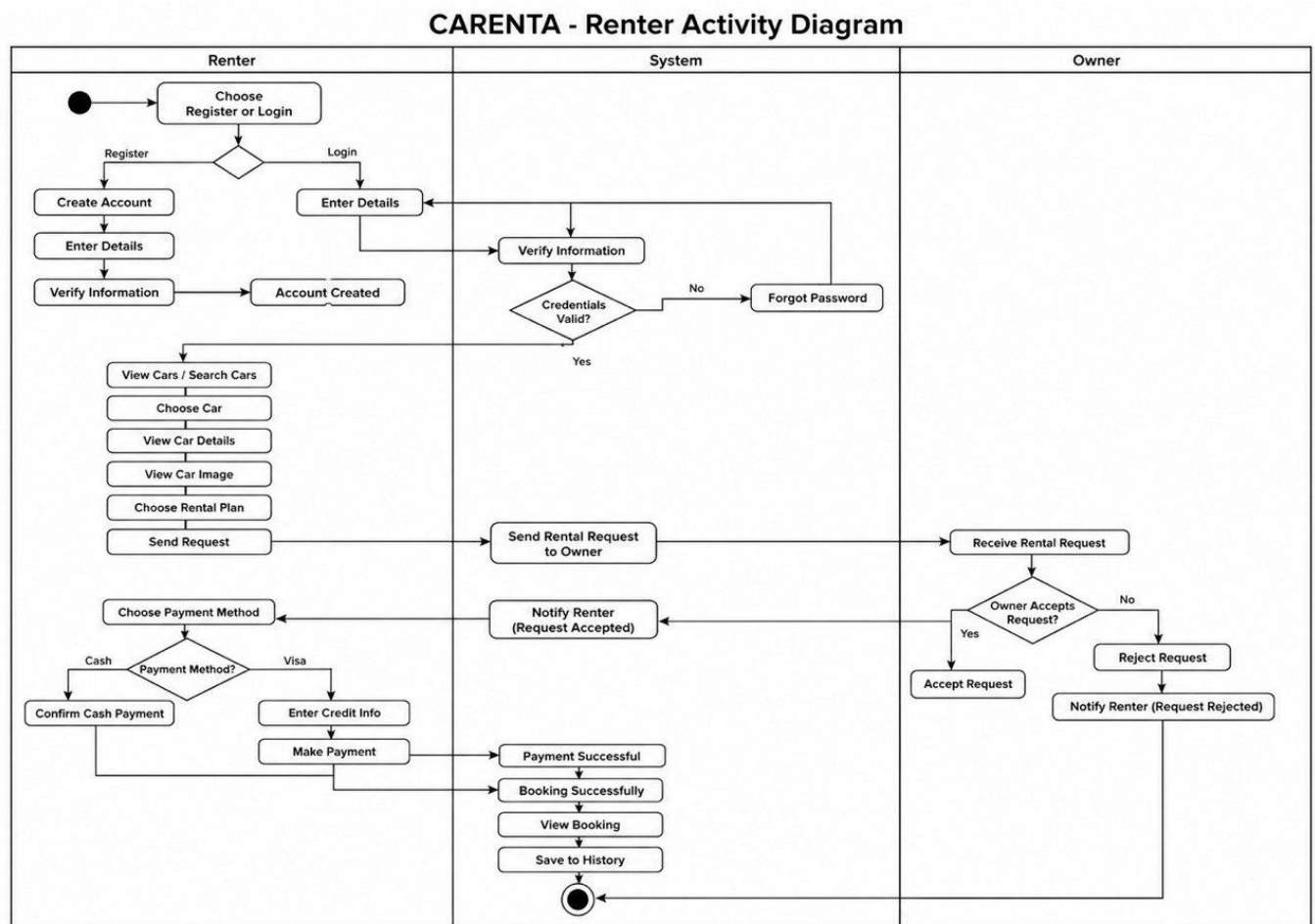
CARENTA - Owner Car Management Sequence Diagram



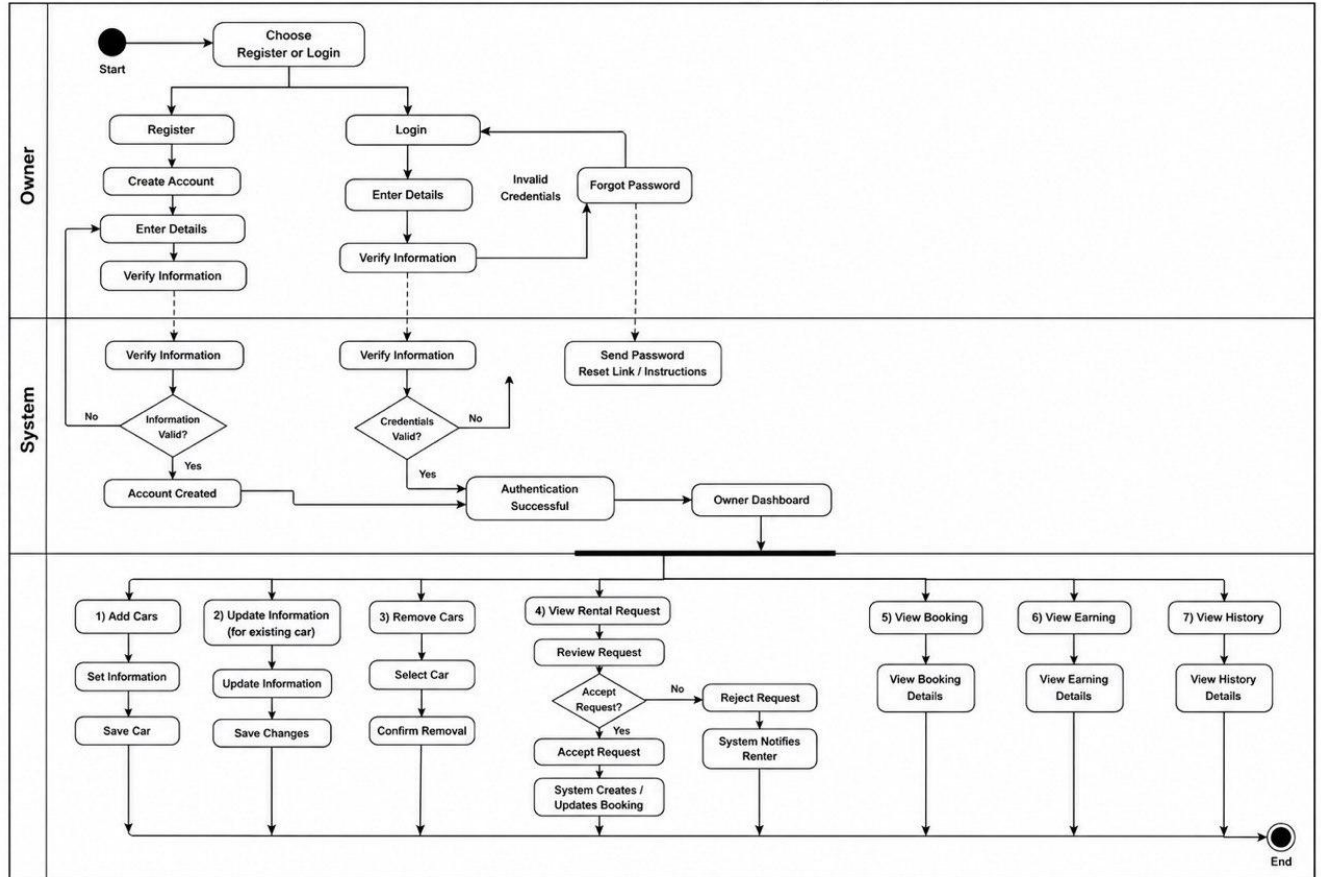
3.6 Activity Diagram

An activity diagram is a graphical representation that depicts the flow of activities or actions within a system. It shows the sequence of actions, decisions, and parallel activities involved in a process or workflow.

The following diagram shows the activity diagram of the CARENTA system



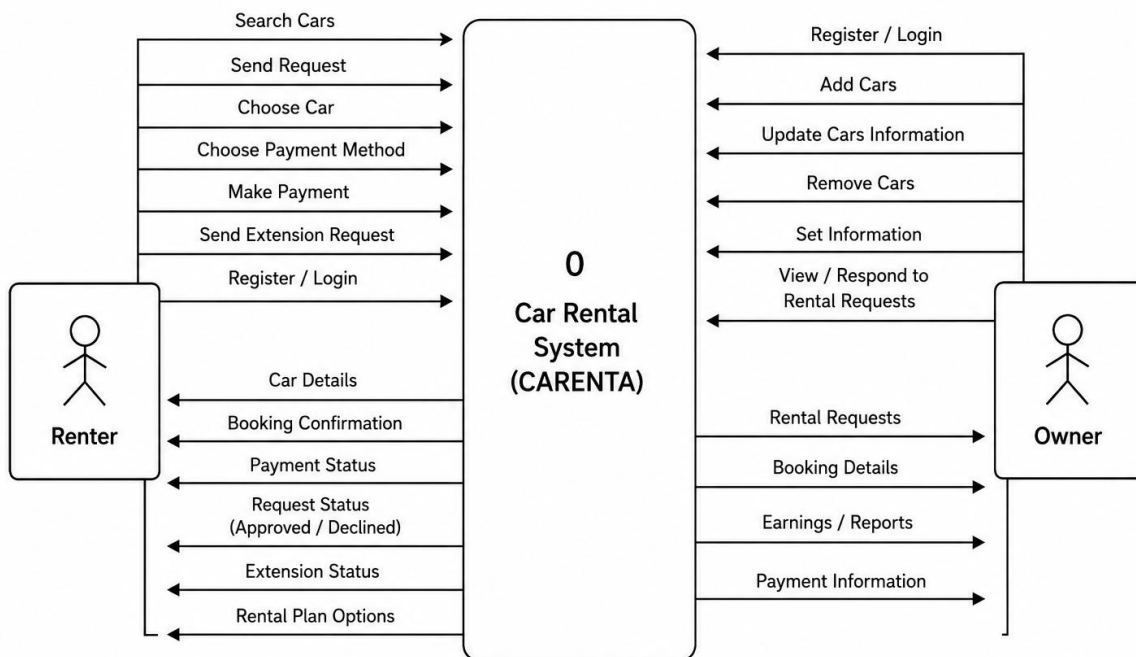
CARENTA - Owner Activity Diagram



3.7 Context Diagram

Context Diagram: The highest level of a DFD, representing the entire system as a single process. It shows how external entities (users, other systems) interact with the system through incoming and outgoing data flows, defining the system's boundaries.

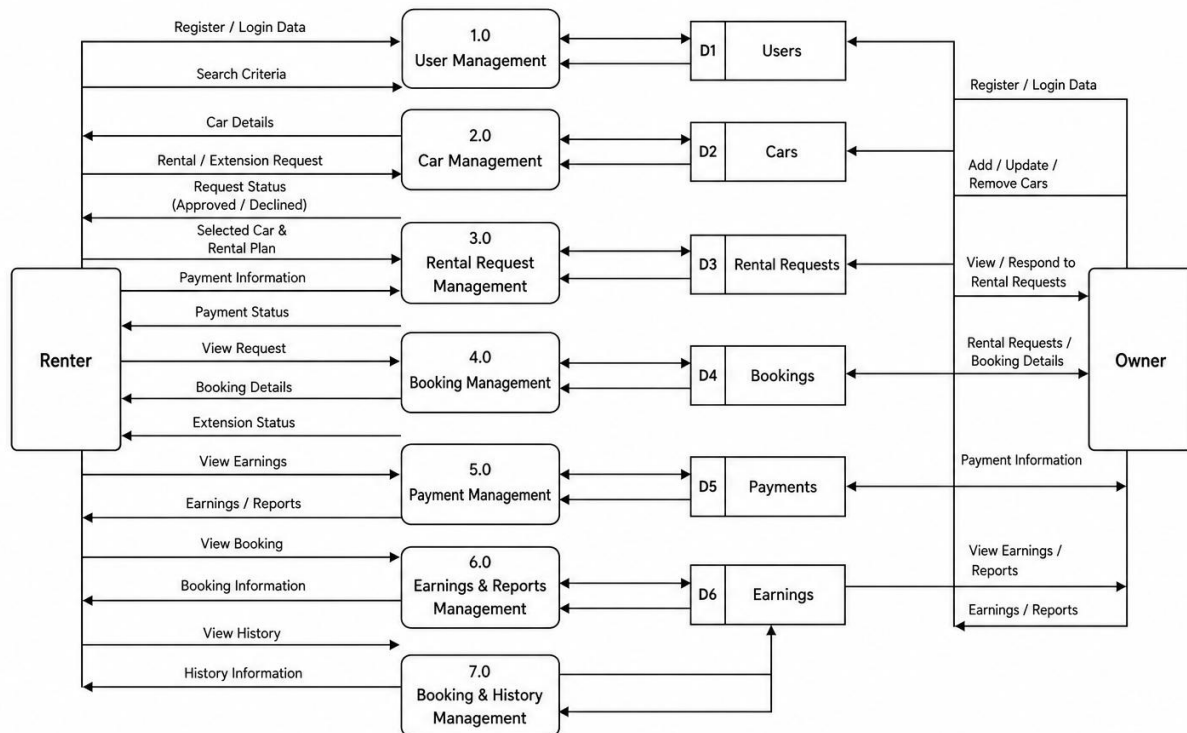
Context Diagram – Car Rental System (CARENTA)



3.8 Data Flow Diagram:

is a graphical technique used to visualize how data moves through an information system, focusing on the flow and transformation of data rather than programming logic. It uses four main components: processes (which transform incoming data into output), data flows (the pathways or arrows representing data movement), data stores (where data is held for later use, like databases), and external entities (sources or destinations outside the system, such as users or other systems).

DFD Level 1 – Car Rental System (CARENTA)





3.9 Use case description:

Use Case: Search Cars

Use Case Name	Search Cars
Scenario	Renter searches for available vehicles.
Triggering Event	Renter enters search criteria.
Brief Description	Finds vehicles matching search filters.
Actors	Renter
Related Use Cases	View Cars
Stakeholders	Renter
Preconditions	• Vehicles available in system.
Postconditions	• Search results displayed successfully.
Flow of Activities	Actor: 1. Enter search criteria. 2. Submit search request. System: 1. Process search criteria. 2. Retrieve matching vehicles. 3. Display search results.
Exception Conditions	• No matching vehicles found. • System error.



Use Case: View Car Details

Use Case Name	View Car Details
Scenario	Renter views vehicle information.
Triggering Event	Renter selects a vehicle.
Brief Description	Displays detailed information about the selected vehicle.
Actors	Renter
Related Use Cases	View Cars
Stakeholders	Renter, Owner
Preconditions	• Vehicle exists in listings.
Postconditions	• Vehicle details displayed successfully.
Flow of Activities	Actor: 1. Select vehicle. 2. View vehicle details. System: 1. Retrieve vehicle information. 2. Display vehicle details and images.
Exception Conditions	• Vehicle unavailable. • System error.



Use Case: Choose Rental Plan

Use Case Name	Choose Rental Plan
Scenario	Renter selects rental duration and plan.
Triggering Event	Renter proceeds with booking.
Brief Description	Allows renter to select a suitable rental plan.
Actors	Renter
Related Use Cases	Send Request
Stakeholders	Renter, Owner
Preconditions	• Vehicle selected.
Postconditions	• Rental plan selected successfully.
Flow of Activities	Actor: 1. Select rental plan. 2. Confirm selected plan. System: 1. Display available plans. 2. Save selected plan.
Exception Conditions	• Invalid plan selected. • System error.



Use Case: Send Request

Use Case Name	Send Request
Scenario	Renter submits a rental request.
Triggering Event	Renter confirms booking request.
Brief Description	Sends rental request to vehicle owner.
Actors	Renter
Related Use Cases	Choose Rental Plan
Stakeholders	Renter, Owner
Preconditions	• Vehicle selected. • Rental plan selected.
Postconditions	• Request sent successfully.
Flow of Activities	Actor: 1. Confirm request. 2. Submit rental request. System: 1. Validate request. 2. Store request. 3. Notify vehicle owner.
Exception Conditions	• Invalid request. • System error.



Use Case: Choose Payment Method

Use Case Name	Choose Payment Method
Scenario	Renter selects preferred payment method.
Triggering Event	Renter proceeds to payment.
Brief Description	Allows selection of payment type.
Actors	Renter
Related Use Cases	Make Payment
Stakeholders	Renter
Preconditions	• Booking approved.
Postconditions	• Payment method selected successfully.
Flow of Activities	Actor: 1. Select payment method. 2. Confirm selection. System: 1. Display available payment methods. 2. Save selected payment method.
Exception Conditions	• Invalid payment method. • System error.



Use Case: Send Extension Request

Use Case Name	Send Extension Request
Scenario	Renter requests an extension of the rental period.
Triggering Event	Renter selects extension option.
Brief Description	Sends extension request to vehicle owner.
Actors	Renter
Related Use Cases	Rental Period Update
Stakeholders	Renter, Owner
Preconditions	• Active rental exists.
Postconditions	• Extension request submitted successfully.
Flow of Activities	Actor: 1. Select extension option. 2. Enter additional rental period. 3. Submit extension request. System: 1. Validate request. 2. Send request to vehicle owner.
Exception Conditions	• Invalid extension period. • System error.



Use Case: Rental Period Update

Use Case Name	Rental Period Update
Scenario	Rental duration is updated after extension approval.
Triggering Event	Owner approves extension request.
Brief Description	Updates rental period and booking information.
Actors	Renter, Owner
Related Use Cases	Send Extension Request, Pay Additional Amount
Stakeholders	Renter, Owner
Preconditions	• Active rental exists. • Extension request approved.
Postconditions	• Rental period updated successfully.
Flow of Activities	Actor: 1. Request extension. 2. Receive approval notification. System: 1. Verify approval. 2. Update rental duration. 3. Recalculate rental cost. 4. Save updated booking details. 5. Display updated information.
Exception Conditions	• Update failed. • Invalid rental record. • System error.



Use Case: Pay Additional Amount

Use Case Name	Pay Additional Amount
Scenario	Renter pays additional charges after extending the rental period.
Triggering Event	Extension request approved and extra payment required.
Brief Description	Allows renter to pay the additional rental amount.
Actors	Renter
Related Use Cases	Send Extension Request, Rental Period Update
Stakeholders	Renter, Owner
Preconditions	• Extension request approved. • Additional amount calculated.
Postconditions	• Additional payment completed successfully.
Flow of Activities	Actor: 1. Review additional charges. 2. Select payment method. 3. Confirm payment. System: 1. Display payment amount. 2. Process payment. 3. Update payment status. 4. Display confirmation message.
Exception Conditions	• Payment failed. • Invalid payment information. • System error.



Use Case: View Rental Request

Use Case Name	View Rental Request
Scenario	Owner views incoming rental requests.
Triggering Event	Owner selects rental requests option.
Brief Description	Displays all pending rental requests.
Actors	Owner
Related Use Cases	Accept Request, Reject Request
Stakeholders	Owner, Renter
Preconditions	• Owner is logged in.
Postconditions	• Rental requests displayed successfully.
Flow of Activities	Actor: 1. Select rental requests. 2. View request details. System: 1. Retrieve requests. 2. Display requests list.
Exception Conditions	• No requests found. • System error.



Use Case: Accept Request

Use Case Name	Accept Request
Scenario	Owner approves a rental request.
Triggering Event	Owner selects accept option.
Brief Description	Confirms rental request.
Actors	Owner
Related Use Cases	View Rental Request
Stakeholders	Owner, Renter
Preconditions	• Rental request exists.
Postconditions	• Request approved successfully.
Flow of Activities	Actor: 1. Select request. 2. Click accept. System: 1. Validate request. 2. Update request status. 3. Notify renter.
Exception Conditions	• Invalid request. • System error.



Use Case: Reject Request

Use Case Name	Reject Request
Scenario	Owner declines a rental request.
Triggering Event	Owner selects reject option.
Brief Description	Rejects rental request.
Actors	Owner
Related Use Cases	View Rental Request
Stakeholders	Owner, Renter
Preconditions	• Rental request exists.
Postconditions	• Request rejected successfully.
Flow of Activities	Actor: 1. Select request. 2. Click reject. System: 1. Validate request. 2. Update request status. 3. Notify renter.
Exception Conditions	• Invalid request. • System error.



Use Case: View History

Use Case Name	View History
Scenario	Owner reviews previous rental activities.
Triggering Event	Owner selects history option.
Brief Description	Displays completed rental records.
Actors	Owner
Related Use Cases	View Booking
Stakeholders	Owner
Preconditions	• Owner is logged in.
Postconditions	• History displayed successfully.
Flow of Activities	Actor: 1. Open history page. 2. Review records. System: 1. Retrieve history data. 2. Display records.
Exception Conditions	• No records found. • System error.



Use Case: View Vehicle

Use Case Name	View Vehicle
Scenario	Renter wants to browse available vehicles.
Triggering Event	Renter opens vehicle page.
Brief Description	Allow renters to view available vehicles.
Actors	Renter
Related Use Cases	Choose Vehicle
Stakeholders	Users, Admin
Preconditions	• Vehicles available in system.
Postconditions	• Vehicle list displayed successfully.
Flow of Activities	Actor: 1. Open vehicle page. 2. Browse available vehicles. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Choose Vehicle

Use Case Name	Choose Vehicle
Scenario	Renter selects a vehicle.
Triggering Event	Renter clicks vehicle.
Brief Description	Allows renter to choose a vehicle.
Actors	Renter
Related Use Cases	View Vehicle, Booking Successfully
Stakeholders	Users, Admin
Preconditions	• Vehicles listed in system.
Postconditions	• Vehicle selected successfully.
Flow of Activities	Actor: 1. Browse vehicle list. 2. Click on desired vehicle. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Booking Successfully

Use Case Name	Booking Successfully
Scenario	Renter confirms booking.
Triggering Event	Renter clicks confirm booking.
Brief Description	Confirms successful booking.
Actors	Renter
Related Use Cases	Choose Vehicle, Make Payment
Stakeholders	Users, Admin
Preconditions	• Vehicle selected. • Renter is logged in.
Postconditions	• Booking confirmed successfully.
Flow of Activities	Actor: 1. Review booking details. 2. Confirm booking. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Make Payment

Use Case Name	Make Payment
Scenario	Renter pays for booking.
Triggering Event	Renter chooses payment.
Brief Description	Processes payment securely.
Actors	Renter
Related Use Cases	Booking Successfully, Choose Payment Method
Stakeholders	Users, Admin
Preconditions	• Booking confirmed. • Payment method selected.
Postconditions	• Payment processed successfully.
Flow of Activities	Actor: 1. Select payment method. 2. Confirm payment. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Register

Use Case Name	Register
Scenario	New user creates account.
Triggering Event	User selects register.
Brief Description	Creates a new account.
Actors	User
Related Use Cases	Login
Stakeholders	Users, Admin
Preconditions	• User has no existing account.
Postconditions	• Account created successfully.
Flow of Activities	Actor: 1. Fill in registration details. 2. Submit registration form. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Login

Use Case Name	Login
Scenario	User accesses account.
Triggering Event	User enters credentials.
Brief Description	Allows access to system.
Actors	User
Related Use Cases	Register
Stakeholders	Users, Admin
Preconditions	• User has a registered account.
Postconditions	• User logged in successfully.
Flow of Activities	Actor: 1. Enter credentials. 2. Submit login form. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Add Vehicle

Use Case Name	Add Vehicle
Scenario	Owner adds vehicle.
Triggering Event	Owner selects add option.
Brief Description	Uploads vehicle listing.
Actors	Owner
Related Use Cases	Remove Vehicle
Stakeholders	Users, Admin
Preconditions	• Owner is logged in.
Postconditions	• Vehicle listed successfully.
Flow of Activities	Actor: 1. Enter vehicle details. 2. Submit listing. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: Remove Vehicle

Use Case Name	Remove Vehicle
Scenario	Owner removes vehicle.
Triggering Event	Owner selects remove option.
Brief Description	Deletes vehicle from listings.
Actors	Owner
Related Use Cases	Add Vehicle
Stakeholders	Users, Admin
Preconditions	• Owner is logged in. • Vehicle exists in listings.
Postconditions	• Vehicle removed successfully.
Flow of Activities	Actor: 1. Select vehicle. 2. Confirm removal. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: View Earning

Use Case Name	View Earning
Scenario	Owner checks earnings.
Triggering Event	Owner opens earnings page.
Brief Description	Displays revenue information.
Actors	Owner
Related Use Cases	View Booking
Stakeholders	Users, Admin
Preconditions	• Owner is logged in.
Postconditions	• Earnings displayed successfully.
Flow of Activities	Actor: 1. Open earnings page. 2. Review revenue data. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



Use Case: View Booking

Use Case Name	View Booking
Scenario	Owner/admin views bookings.
Triggering Event	The booking page opened.
Brief Description	Displays booking records.
Actors	Owner, Admin
Related Use Cases	View Earning, View History
Stakeholders	Users, Admin
Preconditions	• Owner/Admin is logged in.
Postconditions	• Booking records displayed successfully.
Flow of Activities	Actor: 1. Open booking page. 2. Review booking records. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.



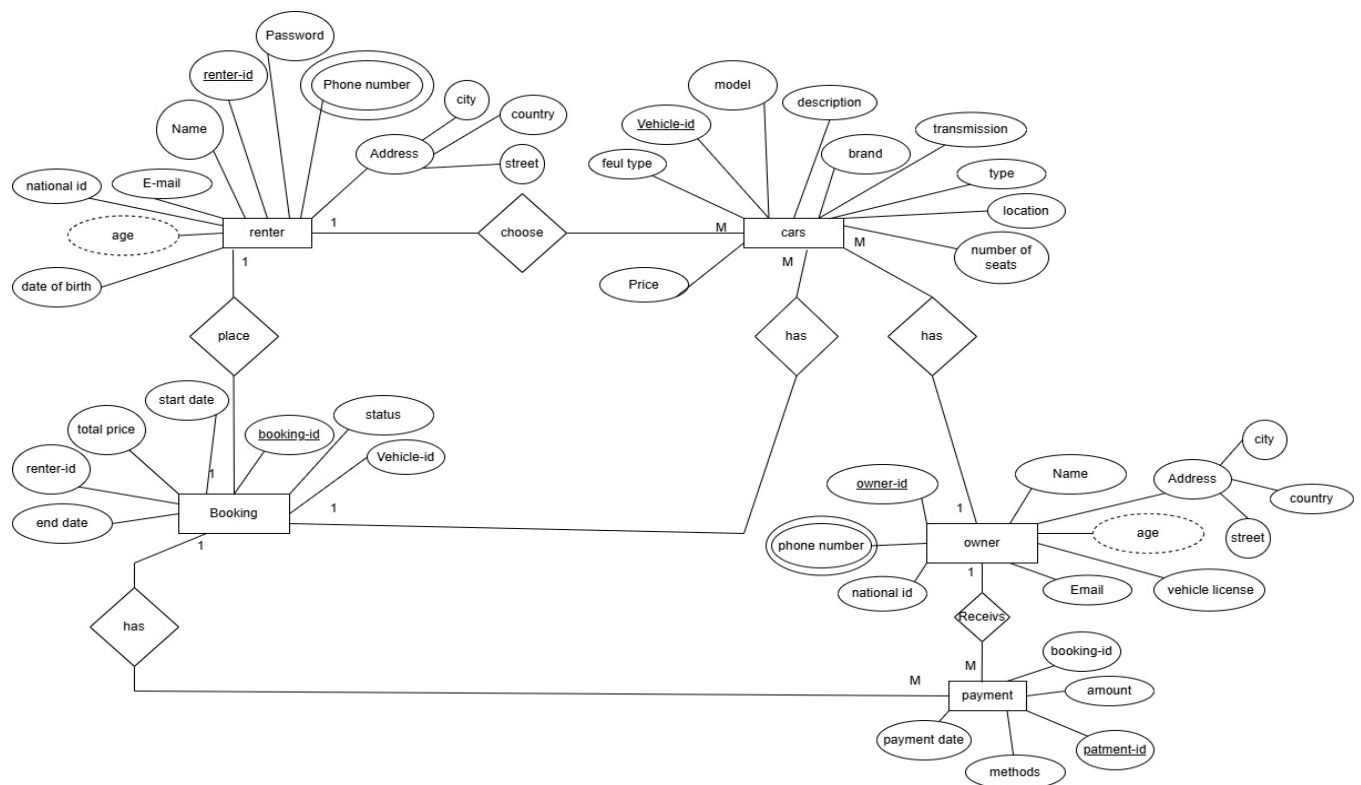
Use Case: Search Cars

Use Case Name	Search Cars
Scenario	Renter searches for available vehicles.
Triggering Event	Renter enters search criteria.
Brief Description	Finds vehicles matching search filters.
Actors	Renter
Related Use Cases	View Vehicle
Stakeholders	Users, Admin
Preconditions	• Vehicles available in system.
Postconditions	• Search results displayed successfully.
Flow of Activities	Actor: 1. Enter search criteria. 2. Submit search request. System: 1. Validate request. 2. Process data. 3. Display results.
Exception Conditions	• Invalid input. • System error.

3.2 System Design

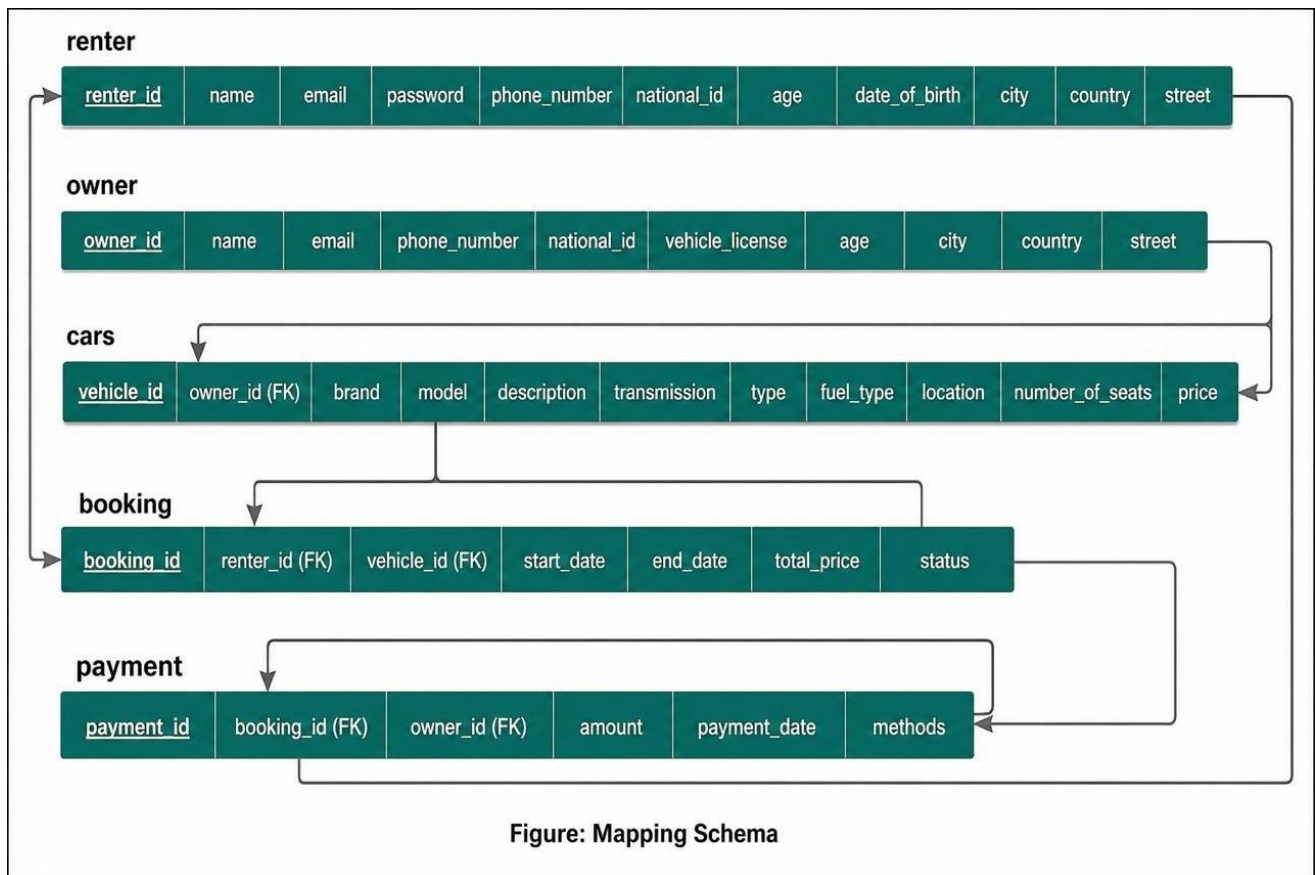
3.2.1 Entity Relationship Diagram

An Entity Relationship Diagram (ERD) is a visual model that represents the logical structure of a database. It illustrates the entities (like objects or concepts) within a system and the relationships between them. Using standard symbols like rectangles and diamonds, it shows how data is stored and connected. It serves as a foundational blueprint for designing and building efficient databases



3.2.2 Mapping Schema:

Creating a mapping schema is essential for structuring and transforming data between different systems, ensuring consistency, accuracy, and seamless integration in data management processes, the following diagram describes the mapping schema designed for the application:







4.1 UI Screens

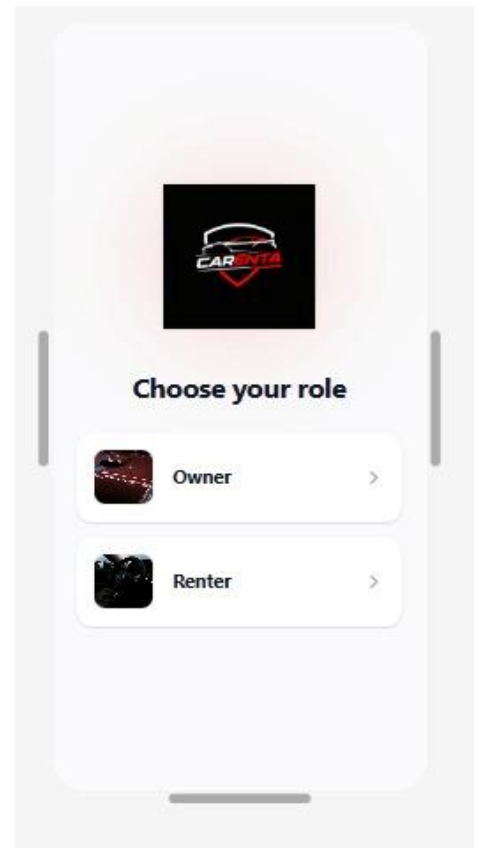
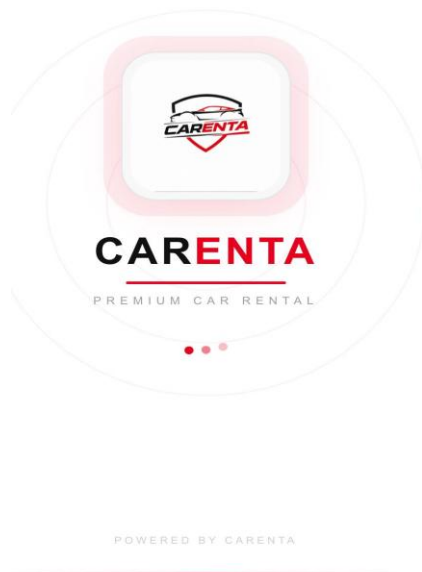


Figure13 The splash screen of CARENTA features a bold red-to-white gradient background that reflects the brand's energetic, trustworthy, and premium identity. At the center, the "CARENTA" wordmark is displayed prominently with a sleek, modern automotive-inspired typography, accompanied by a subtle car silhouette or icon above it to visually communicate the nature of the rental service.

Figure14 Choose Your Role Screen Description: This is the primary gateway screen of the application. User Action: The user selects their specific identity, choosing either Car Owner to manage and rent out vehicles, or Renter to browse and book available cars

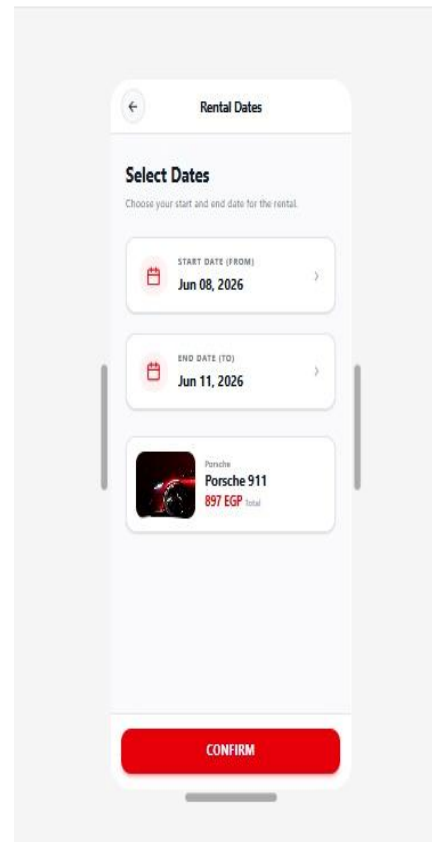
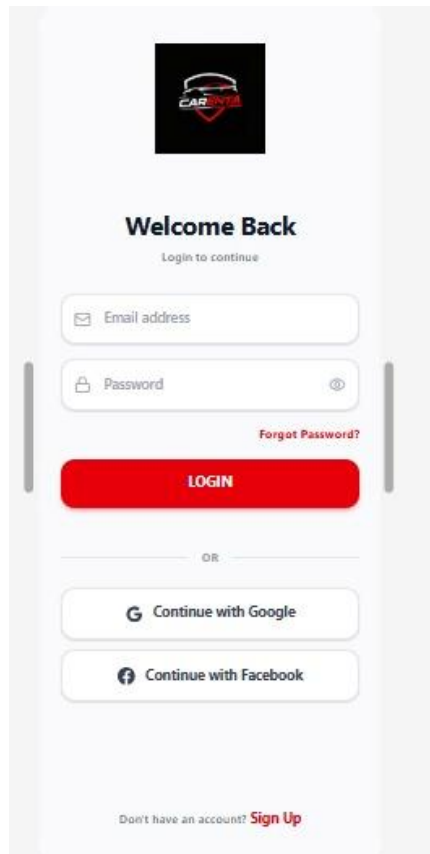


Figure15 Authentication & Login Screen Description: The secure access portal for returning users. User Action: The user inputs their registered email address and password, or utilizes social media login options, to authenticate their identity and access their profile

Figure16 Rental Dates Selection Screen Description: The scheduling and automated invoice calculation interface. User Action: The renter chooses their pickup (start) and return (end) dates via an interactive calendar, reviews the dynamically computed final price, and confirms the booking

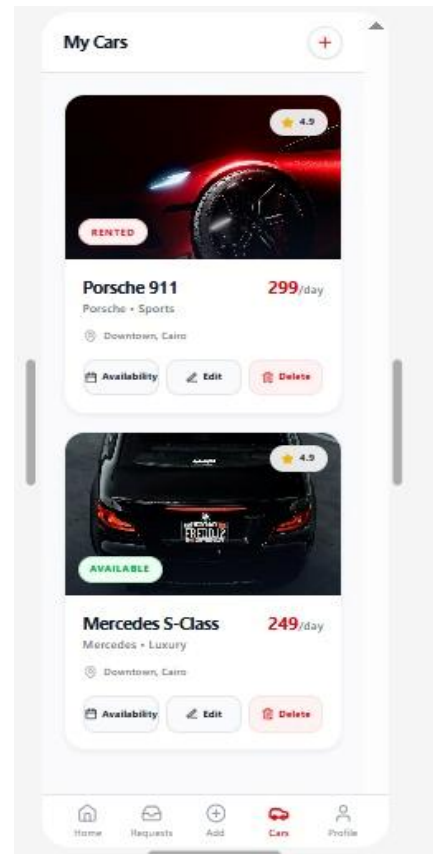
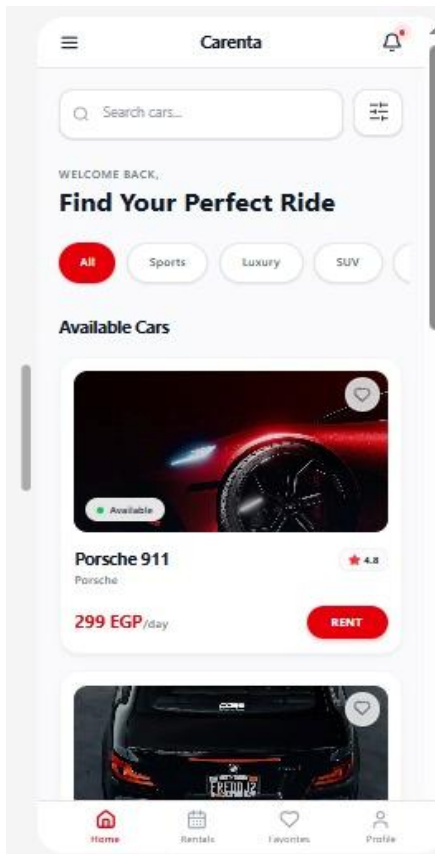
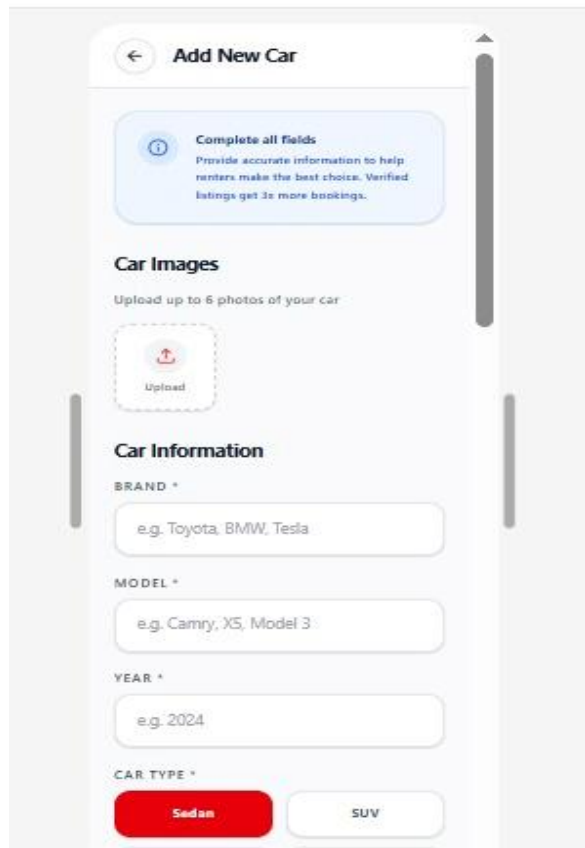


Figure17 Renter Dashboard (Home Screen)Description: The main marketplace feed and discovery hub for renters. User Action: The renter browses available vehicles, utilizes the search bar with advanced filters, sorts cars by categories (Sports, Luxury, SUV), and selects a car to rent

Figure18 Fleet Management (My Cars Screen)Description: The inventory control panel for the owner's fleet. User Action: The owner tracks the real-time status of all their listed vehicles (whether available or rented), reviews individual car ratings, and modifies or deletes listings.



Add New Car

Complete all fields
 Provide accurate information to help renters make the best choice. Verified listings get 3x more bookings.

Car Images
 Upload up to 6 photos of your car

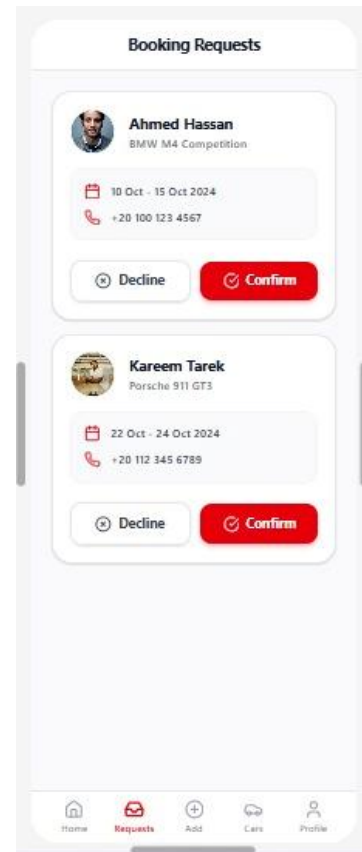
Car Information

BRAND *
 e.g. Toyota, BMW, Tesla

MODEL *
 e.g. Camry, X5, Model 3

YEAR *
 e.g. 2024

CAR TYPE *
 Sedan SUV



Booking Requests

Ahmed Hassan
 BMW M4 Competition
 10 Oct - 15 Oct 2024
 +20 100 123 4567
 Decline Confirm

Kareem Tarek
 Porsche 911 GT3
 22 Oct - 24 Oct 2024
 +20 112 345 6789
 Decline Confirm

Home Requests Add Cars Profile

Figure19 Add New Car (Description) : The multi-step vehicle listing and publishing form. User Action: The owner uploads vehicle images, fills in specifications (model, year, fuel type), defines custom pricing tiers, sets rental rules, and publishes the listing to the public marketplace

Figure20 Booking Requests Screen Description: The transaction management and approval panel for owners. User Action: The owner reviews incoming rental applications from clients, views renter profiles and requested dates, and either accepts or declines each booking.

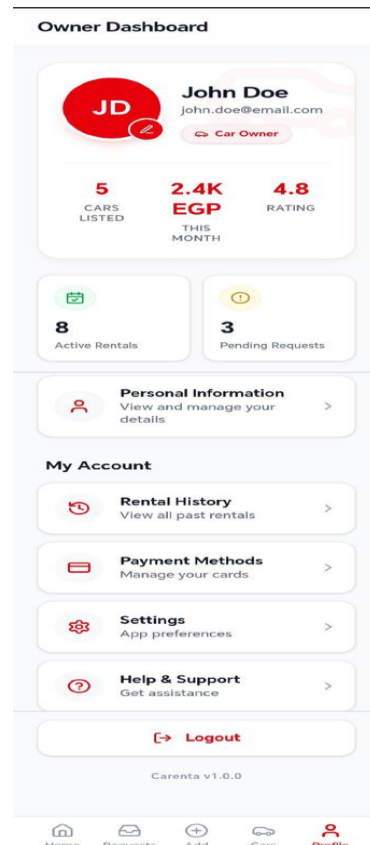
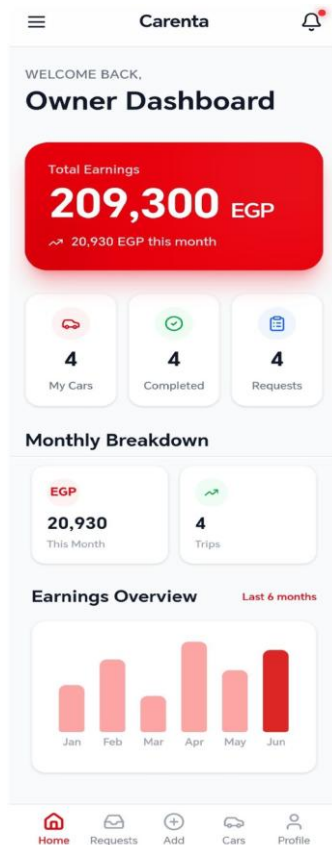
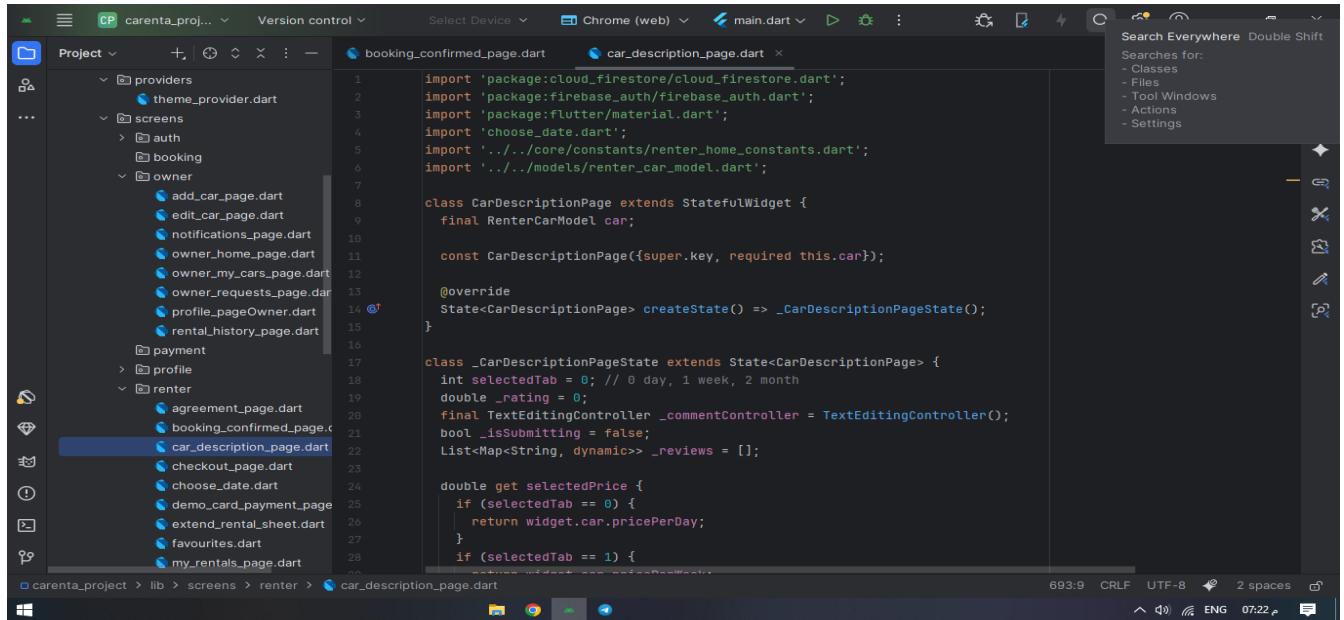


Figure21 Owner Dashboard Screen This screen serves as the main dashboard for car owners in the Carenta application. It provides a quick overview of the owner's performance and activity, including total earnings, monthly income, listed cars, completed rentals, and rental requests. The dashboard also displays a monthly breakdown of earnings and trip statistics, along with a visual chart showing earnings trends over the last six months. The bottom navigation bar allows owners to easily access Home, Requests, Add Car, Cars, and Profile sections

Figure22 The Owner Dashboard provides a centralized view of the car owner's account and rental activities. It displays key information such as profile details, listed vehicles, earnings, ratings, active rentals, and pending requests. The dashboard also offers quick access to personal information, rental history, payment methods, settings, and support services. Additionally, it allows owners to manage their accounts efficiently and securely log out of the application.

4.2 System Code:

Figure 23 Car Description Page



```

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'choose_date.dart';
import '../core/constants/renter_home_constants.dart';
import '../models/renter_car_model.dart';

class CarDescriptionPage extends StatefulWidget {
  final RenterCarModel car;

  const CarDescriptionPage({super.key, required this.car});

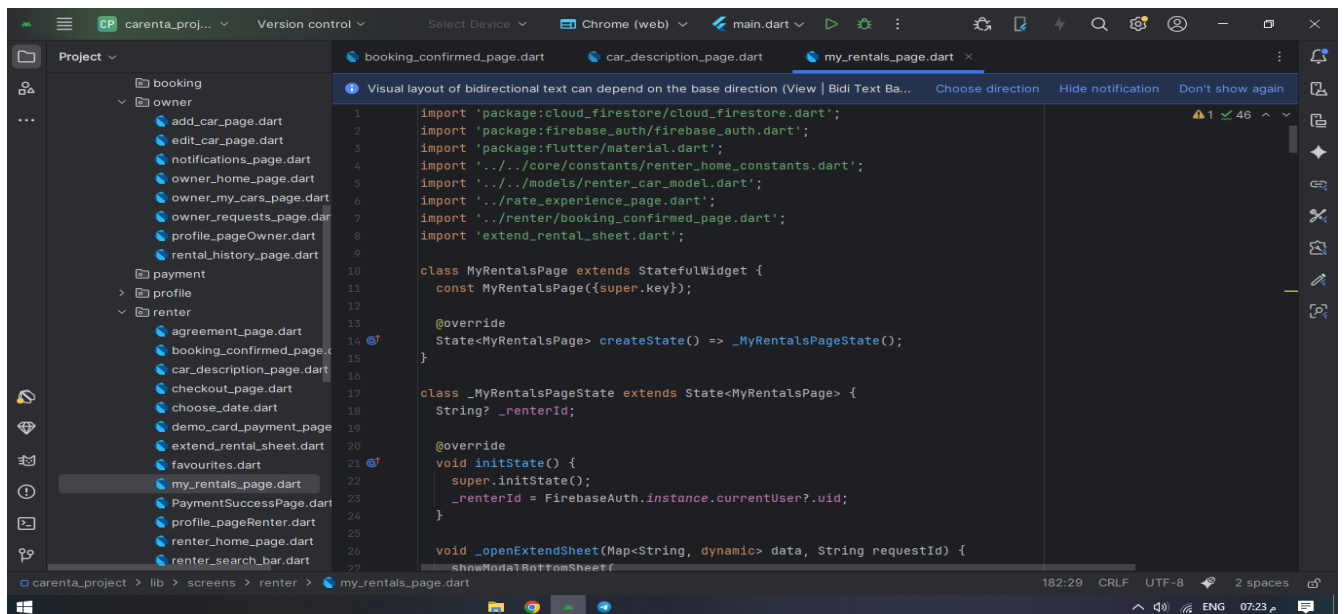
  @override
  State<CarDescriptionPage> createState() => _CarDescriptionPageState();
}

class _CarDescriptionPageState extends State<CarDescriptionPage> {
  int selectedTab = 0; // 0 day, 1 week, 2 month
  double _rating = 0;
  final TextEditingController _commentController = TextEditingController();
  bool _isSubmitting = false;
  List<Map<String, dynamic>> _reviews = [];

  double get selectedPrice {
    if (selectedTab == 0) {
      return widget.car.pricePerDay;
    }
    if (selectedTab == 1) {

```

Figure 24 My Rentals Page



```

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import '../core/constants/renter_home_constants.dart';
import '../models/renter_car_model.dart';
import '../rate_experience_page.dart';
import '../renter/booking_confirmed_page.dart';
import 'extend_rental_sheet.dart';

class MyRentalsPage extends StatefulWidget {
  const MyRentalsPage({super.key});

  @override
  State<MyRentalsPage> createState() => _MyRentalsPageState();
}

class _MyRentalsPageState extends State<MyRentalsPage> {
  String? _renterId;

  @override
  void initState() {
    super.initState();
    _renterId = FirebaseAuth.instance.currentUser?.uid;
  }

  void _openExtendSheet(Map<String, dynamic> data, String requestId) {
    showModalBottomSheet(

```

The screenshot shows an IDE with the following project structure in the file explorer:

- Project
 - booking
 - owner
 - add_car_page.dart
 - edit_car_page.dart
 - notifications_page.dart
 - owner_home_page.dart
 - owner_my_cars_page.dart
 - owner_requests_page.dart
 - profile_pageOwner.dart
 - rental_history_page.dart
 - payment
 - profile
 - renter
 - agreement_page.dart
 - booking_confirmed_page.dart
 - car_description_page.dart
 - checkout_page.dart
 - choose_date.dart
 - demo_rental_payment_page.dart
 - extend_rental_sheet.dart
 - favourites.dart
 - my_rentals_page.dart
 - PaymentSuccessPage.dart
 - profile_pageRenter.dart
 - renter_home_page.dart
 - renter_search_bar.dart

The main editor displays the code for `add_car_page.dart`:

```
import '../widgets/owner/add_car/car_added_success_sheet.dart';
import '../widgets/owner/bottom_bar_owner.dart';
import '../core/constants/renter_home_constants.dart';

class AddCarPage extends StatefulWidget {
  const AddCarPage({super.key});

  @override
  State<AddCarPage> createState() => _AddCarPageState();
}

class _AddCarPageState extends State<AddCarPage> {
  final _formKey = GlobalKey<FormState>();
  final CarService _carService = CarService();

  final _brandController = TextEditingController();
  final _modelController = TextEditingController();
  final _yearController = TextEditingController();
  final _seatsController = TextEditingController();
  final _pricePerDayController = TextEditingController();
  final _pricePerWeekController = TextEditingController();
  final _pricePerMonthController = TextEditingController();
  final _descriptionController = TextEditingController();
  final _minAgeController = TextEditingController();
  final _additionalRulesController = TextEditingController();

  String _carType = 'Sedan';
  String _transmission = 'Automatic';
  String _fuelType = 'Gasoline';
}
```

The screenshot shows an IDE with a project named 'carenta_project'. The file explorer on the left shows the project structure, including a 'services' folder containing 'auth_service.dart'. The main editor displays the code for 'auth_service.dart', which includes imports for 'firebase_auth' and 'firebase_firestore'. The code defines a 'createSocialUserDocIfNeeded' function that takes a 'User' object and a 'UserRole' enum as input. It checks if a document with the user's UID exists in the 'owners' collection. If not, it creates a new document in the 'renters' collection with fields for 'uid', 'name', 'email', 'phone', 'photoUrl', 'role', 'loginMethod', 'emailVerified', and 'createdAt'.

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_firestore/firebase_firestore.dart';

Future<void> createSocialUserDocIfNeeded({
  required firebase_auth.User user,
  required String userRole,
  required String loginMethod,
}) async {
  final uid = user.uid;

  final ownerDoc = await _firebaseFirestore.collection('owners').doc(uid).get();
  final renterDoc = await _firebaseFirestore.collection('renters').doc(uid).get();

  if (!ownerDoc.exists && !renterDoc.exists) {
    final collection = _collectionByRole(userRole);

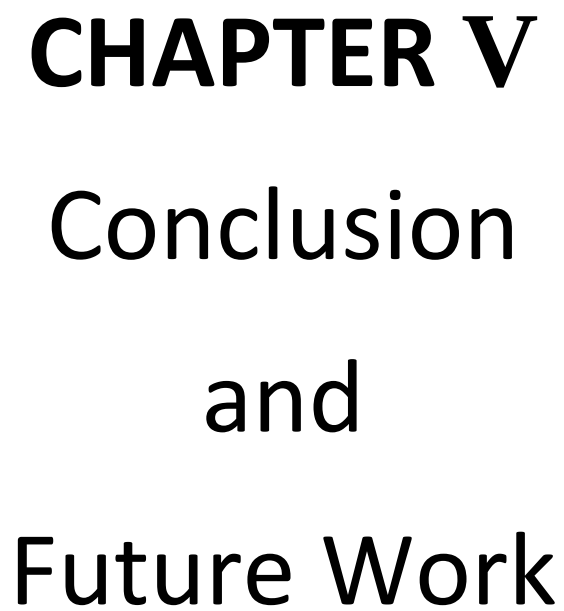
    await _firebaseFirestore.collection(collection).doc(uid).set({
      'uid': uid,
      'name': user.displayName ?? '',
      'email': user.email ?? '',
      'phone': '',
      'photoUrl': user.photoURL ?? '',
      'role': userRole,
      'loginMethod': loginMethod,
      'emailVerified': user.emailVerified,
      'createdAt': FieldValue.serverTimestamp(),
    });
  }
}

```



The screenshot shows an IDE with the following components:

- Top Bar:** Displays the current project path as `carenta_proj...`, the version control status as `Version control`, and the active window as `main.dart`.
- Left Panel (Project Explorer):** Shows the project structure with folders like `services` and `utils`, and files like `main.dart`.
- Main Editor:** Displays the content of `main.dart`, which includes:
 - Imports for `firebase_core`, `firebase_core.dart`, `flutter/material`, `provider/provider`, `supabase_flutter/supabase_flutter`, `firebase_options`, `providers/theme_provider`, and `screens/auth/choose_user_page`.
 - A `main()` function that initializes the app with Firebase and Supabase, sets the theme, and runs the app.
 - A `CarentaApp` class that extends `StatelessWidget` and implements the `Widget` interface.
- Bottom Bar:** Shows the file system path `carenta_project > lib > main.dart` and the current editor settings: `24:2 CRLF UTF-8 2 spaces`.





5.1 Conclusion

CARENTA provides a modern and efficient solution for car rental services by connecting vehicle owners with customers through a user-friendly mobile application. The system simplifies the rental process by offering real-time car availability, secure booking management, and flexible rental options. Additionally, CARENTA delivers significant benefits to both customers and car owners. Customers are able to easily search, compare, and book vehicles that suit their needs, while car owners can increase the utilization of their vehicles and manage their listings in a more organized and effective way. This leads to saving time, reducing manual effort, and improving overall accessibility to rental services. The application also enhances transparency and trust between both parties by providing clear vehicle details, structured booking records, and smooth communication features. These improvements help reduce misunderstandings and ensure a more reliable rental experience. From a technical perspective, CARENTA is built using modern technologies such as Flutter, Firebase, and Supabase, which ensures high performance, scalability, and reliability. The integration of these technologies allows the system to handle real-time updates, secure data storage, and smooth user interaction across different devices. Overall, CARENTA successfully demonstrates how mobile applications can transform traditional car rental systems into more efficient, digital, and user-centered platforms. It addresses real-world transportation needs by offering a practical, scalable, and convenient solution that improves the experience for both renters and vehicle owners.



5.2 Future Work

Future enhancements of CARENTA can further improve user experience, system efficiency, and overall functionality. These improvements may include:

In-App Chat System

This feature would allow direct communication between vehicle owners and renters within the application. It improves transparency, helps users clarify booking details quickly, and reduces misunderstandings by enabling real-time messaging.

Multi-Region Support

Expanding the system to support multiple regions or cities would allow CARENTA to serve a larger user base. This includes adapting the platform to different locations, currencies, and availability rules, making the application more scalable and widely usable.

Maps Integration

Integrating mapping services would help users easily locate available cars and nearby pickup/drop-off points. It can also support route guidance and improve the overall convenience of the rental process.

Loyalty and Rewards Program

This feature would introduce a reward system for frequent users, such as points, discounts, or special offers. It encourages customer retention and increases user engagement with the platform.

.

Digital Rental Agreement

This feature enables the creation and management of electronic rental contracts within the application. It includes all essential terms and conditions, allowing both parties to review and accept agreements digitally, with secure storage for future reference.

Smart Notifications

Smart notifications provide users with timely updates such as booking confirmations, reminders before pickup or return time, payment alerts, and promotional offers. This improves user experience by keeping both owners and renters informed in real time.



5.3 References

Google Firebase. (2026). *Firestore Documentation*. Retrieved from <https://firebase.google.com/docs>

[Flutter Documentation](#). (2026). *Flutter Documentation*.

[FlutterFlow Documentation](#). (2026). *FlutterFlow Documentation*.

[Supabase Documentation](#). (2026). *Supabase Documentation*.

[Figma Help Center](#). (2026). *Figma Documentation*.

[Microsoft Visio Support](#). (2026). *Microsoft Visio Documentation*.

[OpenAI ChatGPT Documentation](#). (2026). *ChatGPT Overview and Documentation*.

Turo. (2026). *Official Website*. Retrieved from <https://turo.com>

Getaround. (2026). *Official Website*. Retrieved from <https://getaround.com>

Zipcar. (2026). *Official Website*. Retrieved from <https://www.zipcar.com>

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

Dennis, A., Wixom, B. H., & Roth, R. M. (2018). *Systems Analysis and Design* (7th ed.). Wiley.

Laudon, K. C., & Laudon, J. P. (2022). *Management Information Systems: Managing the Digital Firm* (17th ed.). Pearson.

Stair, R., & Reynolds, G. (2021). *Principles of Information Systems* (14th ed.). Cengage Learning.

Where to cite them

Introduction, Problem Definition, Existing System → Laudon & Laudon (2022), Stair & Reynolds (2021).

Methodology (Agile/Scrum) → Pressman & Maxim (2020), Sommerville (2016).

System Analysis, Use Cases, UML, ERD, DFD → Dennis et al. (2018).

Flutter, Firebase, Supabase, FlutterFlow, Figma → Official documentation references.

Previous Applications (Turo, Getaround, Zipcar) → Their official websites.

ChatGPT tool description → OpenAI documentation.